# LAB-20 APP DEPLOYMENT USING ECS AND ECR

**Name:- Vansh Bhatt**

**SapID:- 500125395**

**R.No:- R2142231689**

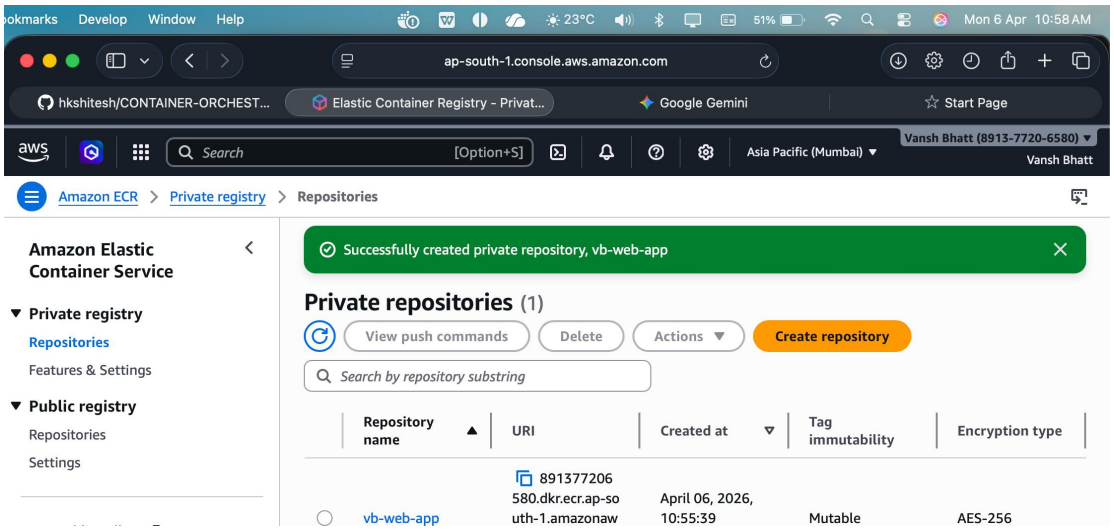
**Batch:- DevOps - B1**

**To:- Hitesh Sharma Sir**

## Phase 1: Prepare and Push to Amazon ECR

ECR is your private library for Docker images.

**Create an ECR Repository:** In the AWS Console (or via CLI), create a repository named after your app (e.g., `my-web-app`).




**Authenticate Docker to ECR:** Run the login command provided by the AWS console (it usually looks like this):

Bash

```
aws ecr get-login-password --region <your-region> | docker login --
username AWS --password-stdin <aws_account_id>.dkr.ecr.<your-
region.amazonaws.com
```

```
vanshssupermac@Vanshs-MacBook-Air-5 Lab20 ECS and ECR % aws ecr get-login-password --region ap-south-
1 | docker login --username AWS --password-stdin 891377206580.dkr.ecr.ap-south-1.amazonaws.com
Login Succeeded
vanshssupermac@Vanshs-MacBook-Air-5 Lab20 ECS and ECR %
```

**Build and Tag your Image:** Build your local Docker image and tag it with the ECR repository URI.

Bash

```
docker build -t my-web-app .
```

```
24°C  49%  Mon 6 Apr 11:10 AM
-zsh
vanshssupermac@Vanshs-MacBook-Air-5 my-web-app % docker build -t my-web-app .
[+] Building 56.5s (9/9) FINISHED                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                  0.0s
 => => transferring dockerfile: 102B                                                  0.0s
 => [internal] load metadata for docker.io/library/httpd:latest                       5.8s
 => [auth] library/httpd:pull token for registry-1.docker.io                          0.0s
 => [internal] load .dockerignore                                                     0.0s
 => => transferring context: 2B                                                       0.0s
 => [1/3] FROM docker.io/library/httpd:latest@sha256:331548c5249bdeced0f048bc2fb8c6b6427d2ec6  50.4s
 => => resolve docker.io/library/httpd:latest@sha256:331548c5249bdeced0f048bc2fb8c6b6427d2ec65  0.0s
 => => sha256:5b51c37387b073841d96ddfcfddc0ea16efb1c55bd1dc411d51c76ea52cada34 288B / 288B     1.1s
 => => sha256:fa78f900eab366f51bc2b4e5ea27bc30426cdac6a1ac424d1c6e8f9314394 13.38MB / 13.38MB  28.9s
 => => sha256:08a23f9aff86409e01971162345bff69d6e8b2d36c8dace4b5456230141779f 1.97MB / 1.97MB  25.5s
 => => sha256:aa0f76b8d9b1aa3aa97e3acb87a8f40cd4f0739d7072705d8388efca4330bc5d 145B / 145B      8.4s
 => => sha256:f4badedbec24858ef2dc51256f6418328e305e9c3c5a5e093581f83425618 30.14MB / 30.14MB  48.5s
 => => extracting sha256:f4badedbec24858ef2dc51256f6418328e305e9c3c5a5e093581f83425618bd5      0.5s
 => => extracting sha256:aa0f76b8d9b1aa3aa97e3acb87a8f40cd4f0739d7072705d8388efca4330bc5d      0.0s
 => => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1      0.0s
 => => extracting sha256:08a23f9aff86409e01971162345bff69d6e8b2d36c8dace4b5456230141779ff      0.0s
 => => extracting sha256:fa78f900eab366f51bc2b4e5ea27bc30426cdac6a1ac424d1c6e8f931439464b      0.2s
 => => extracting sha256:5b51c37387b073841d96ddfcfddc0ea16efb1c55bd1dc411d51c76ea52cada34      0.0s
 => [internal] load build context                                                     0.0s
 => => transferring context: 180B                                                     0.0s
 => [2/3] WORKDIR /usr/local/apache2/htdocs/                                          0.2s
 => [3/3] COPY index.html .                                                           0.0s
 => exporting to image                                                                0.1s
 => => exporting layers                                                               0.0s
 => => exporting manifest sha256:c4cd20a5214925c55cd1c90e262972b9bcfe95abff90a11e0f7558caf9ead  0.0s
 => => exporting config sha256:0c3791a0c5d84f604312dd627ca43badee5f5cd16c2447a2930e6212493e9f5  0.0s
 => => exporting attestation manifest sha256:df9976e313b2dd80ec50a86bf17b1239261f5ed9621a08efa  0.0s
 => => exporting manifest list sha256:a28cd95fd23f0b2ba3c2ef47765b8702c6b14e764bac54afb9f1777d  0.0s
 => => naming to docker.io/library/my-web-app:latest                                  0.0s
 => => unpacking to docker.io/library/my-web-app:latest                               0.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/uklhf0cse5jxbx2ljwzu
1ni6s
vanshssupermac@Vanshs-MacBook-Air-5 my-web-app %
```

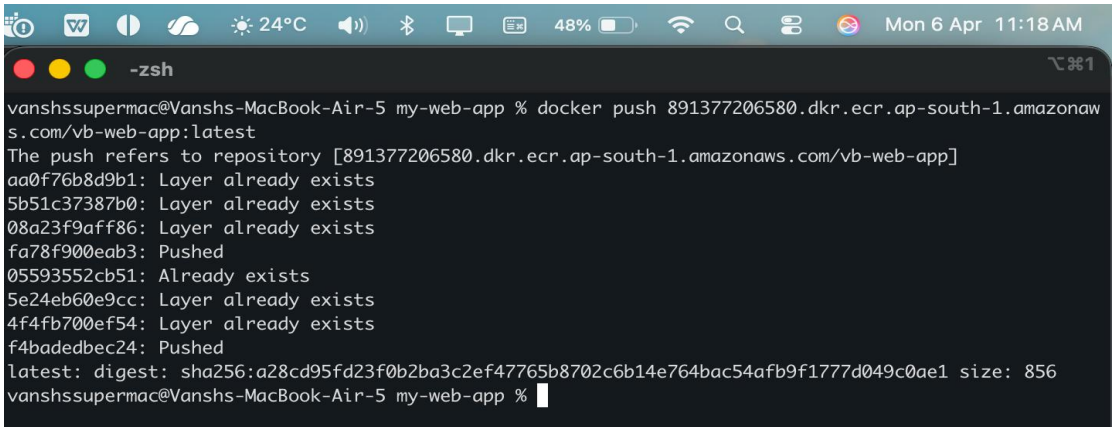
```
docker tag my-web-app:latest <aws_account_id>.dkr.ecr.<your-
region>.amazonaws.com/my-web-app:latest
```

```
24°C  49%  Mon 6 Apr 11:11 AM
-zsh
vanshssupermac@Vanshs-MacBook-Air-5 my-web-app % docker tag my-web-app:latest 891377206580.dkr.ecr.ap
-south-1.amazonaws.com/vb-web-app:latest
vanshssupermac@Vanshs-MacBook-Air-5 my-web-app %
```

**Push the Image:**

Bash

```
docker push <aws_account_id>.dkr.ecr.<your-region>.amazonaws.com/my-web-app:latest
```

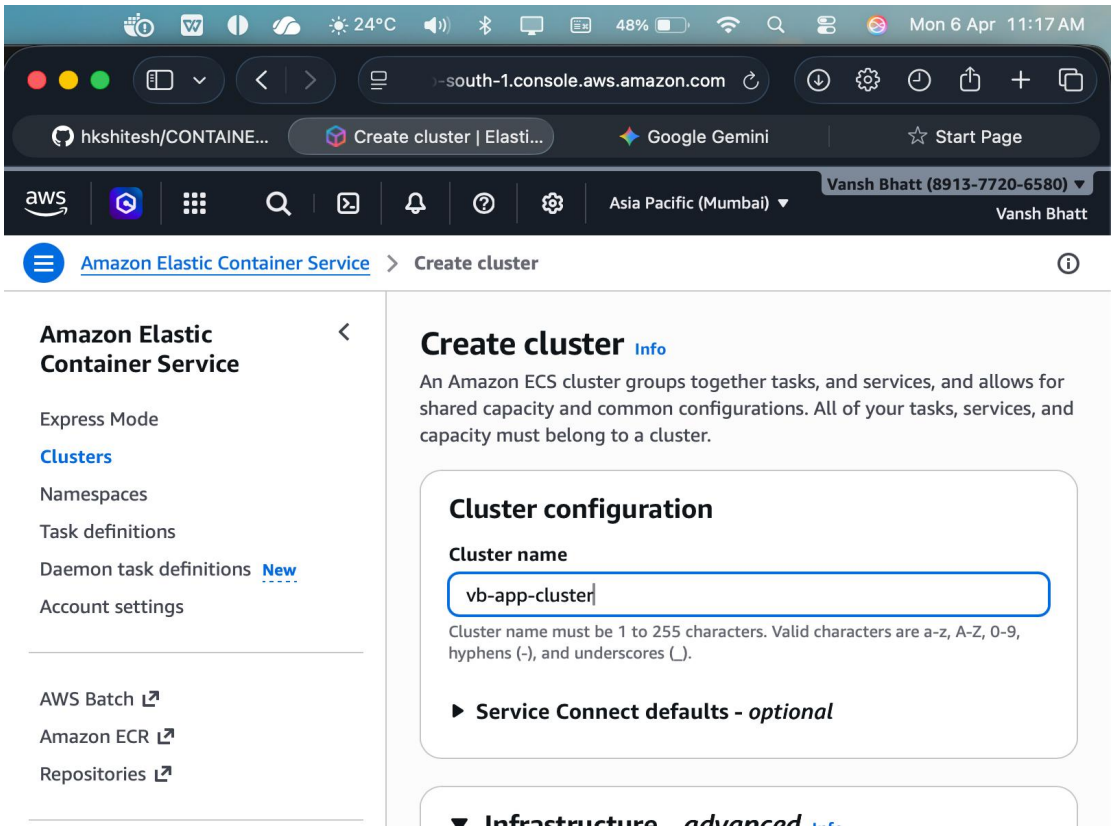



# Phase 2: Set up Amazon ECS

ECS manages where and how your containers run.

## 1. Create a Cluster

A cluster is a logical grouping of tasks or services. In the ECS console, choose **Create Cluster**. For most modern apps, select **Fargate (Serverless)**—this means you don't have to manage underlying EC2
servers.



## 2. Create a Task Definition

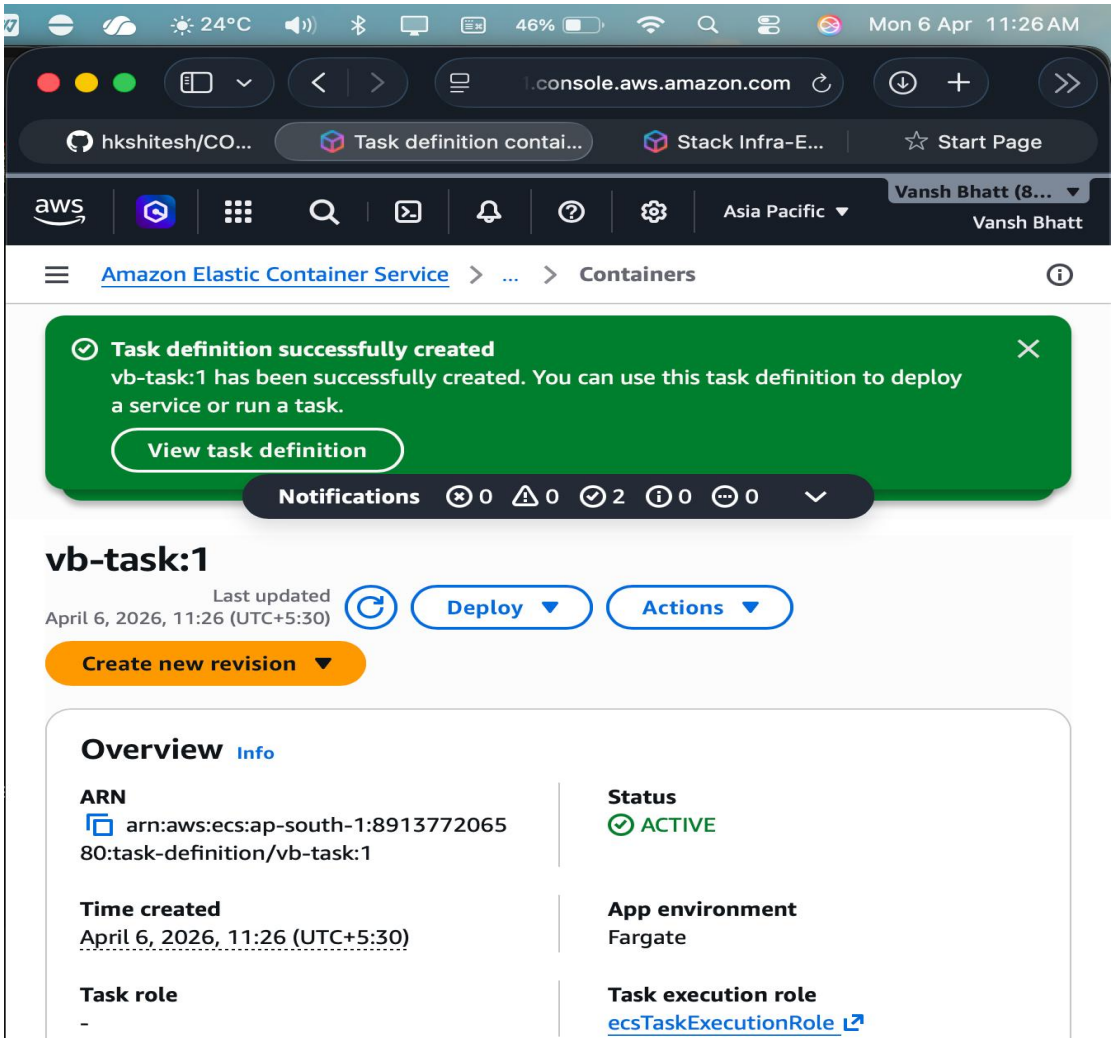
Think of the **Task Definition** as the "blueprint" for your app.

**Launch Type:** Fargate.

**Container Image:** Paste the ECR Image URI you pushed in Phase 1.

**Port Mappings:** Usually port `80` or `8080` (whatever your app listens on).

**Resources:** Assign CPU and Memory (e.g., 0.5 vCPU and 1 GB RAM).



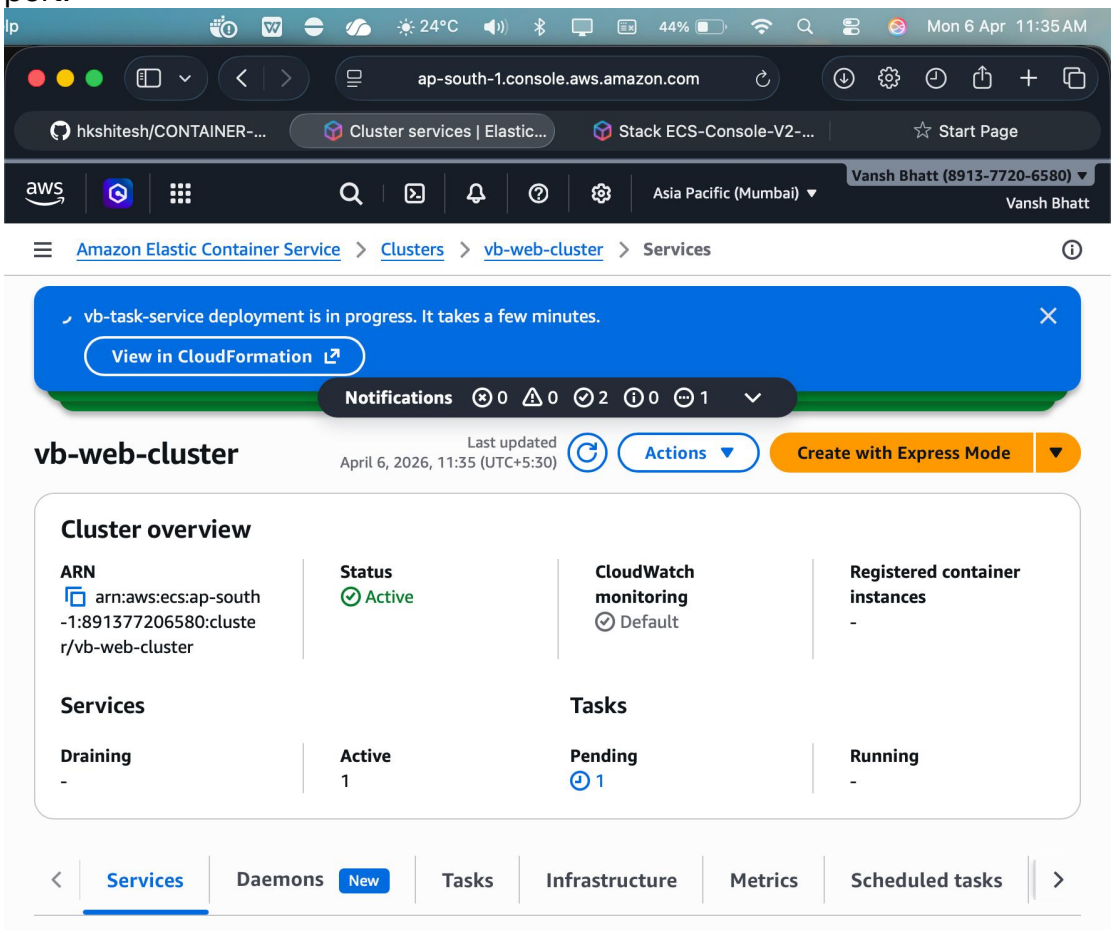

## 3. Create a Service

A **Service** tells ECS how many copies of your Task Blueprint to keep running.

**Desired Tasks:** Set this to `2` or more for high availability.

**Load Balancing:** (Recommended) Select an **Application Load Balancer (ALB)**. This provides a single URL to access your app and distributes traffic between your running containers.

**Networking:** Ensure your VPC and Security Groups allow traffic on your application's port.




# Phase 3: Access and Update

**Verification:** Once the service status says "Active" and the tasks are "Running," go to the **Load Balancing** section to find your **DNS Name**. Paste that into your browser to see your app.

**Updating:** To deploy a new version, simply push a new image to ECR with the same tag (or a new one), update the Task Definition with the new image URI, and update the Service to use the latest revision. ECS will automatically perform a **Rolling Update**, replacing old containers with new ones without downtime.

## Quick Comparison: EC2 vs. Fargate

| Feature | EC2 Launch Type | Fargate Launch Type |
|---|---|---|
| **Management** | You manage the servers | AWS manages the servers (Serverless) |

| Feature | EC2 Launch Type | Fargate Launch Type |
| --- | --- | --- |
| **Cost** | Pay for the instances | Pay for CPU/RAM per second |
| **Complexity** | High (patching, scaling) | Low (just provide the container) |

# THANK YOU!